Constraint Satisfaction Problems Study Notes

Part 1: Introduction to Constraint Satisfaction Problems

Learning Goal

By the end of this part, you should understand what a Constraint Satisfaction Problem (CSP) is, how it differs from a standard search problem, and why this representation is powerful in artificial intelligence problem solving.

1. Core Concept

A Constraint Satisfaction Problem is a problem in which we must assign values to a fixed set of variables while obeying a set of restrictions called constraints.

Every CSP has three main components:

- **Variables:** the unknowns we need to assign values to.
- **Domains:** the set of possible values each variable can take.
- **Constraints:** the rules that decide which combinations of values are allowed.

A solution to a CSP is a **complete assignment** of values to all variables such that **all constraints are satisfied**.

> **Things to Note:**
>
> - A CSP is not just about finding any path. It is about finding a valid final assignment.
> - The final answer must assign values to all variables and satisfy every hard constraint.
> - If even one hard constraint is violated, the assignment is not a valid solution.

2. Why CSPs Are Different from Standard Search Problems

In a standard search problem, a state can be represented in almost any way, as long as we can test goals, generate successors, and evaluate progress. That makes standard search flexible, but it often hides the internal structure of the problem.

In a CSP, the structure is made explicit. Instead of treating the state as a black box, the problem is written directly in terms of variables, domains, and constraints.

This makes a big difference because once structure is visible, specialized algorithms can use that structure to solve problems more efficiently than general-purpose search.

Standard Search Problem	Constraint Satisfaction Problem
State can be any representation.	State is defined through variables and their domains.
Goal test checks whether a target state has been reached.	Goal test checks whether all constraints are satisfied.
Path often matters.	Usually, only the final assignment matters.
Structure may remain hidden.	Structure is explicit and can be exploited algorithmically.

Common Confusion:

Students often assume that CSP is just another name for any search problem. That is wrong. A CSP is a special kind of search problem where the problem is represented in a structured form, and that structure is exactly what makes stronger solving methods possible.

3. Understanding the Three Main Components

Variables

Variables are the entities that need values. In different problems, variables can represent regions, times, jobs, rooms, people, digits, or positions.

Domains

The domain of a variable is the set of values it is allowed to take. Some domains are small and finite, while others can be large, continuous, or even infinite.

Constraints

Constraints define which assignments are legal. They restrict the combinations of values that variables can take together.

Basic Rule:

If variables are the unknowns and domains are the possibilities, then constraints are the filters that remove illegal possibilities.

4. Worked Understanding Through Two Examples

Example A: Map Coloring

Suppose we want to color a map so that neighboring regions do not share the same color.

- **Variables:** the regions
- **Domains:** possible colors such as red, green, and blue
- **Constraints:** neighboring regions must have different colors

Here, a solution is a complete coloring of all regions where no two adjacent regions share the same color.

Example B: Exam Scheduling

Suppose a university must schedule exams for four courses.

- **Variables:** `CS101`, `CS102`, `MATH201`, `ENG101`
- **Domains:** `{Mon-AM, Mon-PM, Tue-AM}`
- **Constraints:**
  - `CS101 ≠ CS102`
  - `CS101 ≠ MATH201`
  - `CS102 ≠ ENG101`
  - `ENG101 ≠ Mon-AM`

Here, a solution is a complete assignment of time slots to all exams such that no hard constraint is broken.

Problem	Variables	Domains	Constraints
Map Coloring	Regions	Colors	Adjacent regions must differ
Exam Scheduling	Exams	Time slots	Conflicting exams must differ; some slots may be forbidden

Things to Think About:

- Why can both map coloring and exam scheduling be represented in the same formal way even though they look very different in real life?
- What changes when you model a problem using variables, domains, and constraints instead of informal description?
- If a problem has many restrictions but no clear variables and domains yet, what should you identify first?

5. Types of Constraints

Constraints can involve different numbers of variables.

Unary Constraints

A unary constraint involves only one variable.

Example from exam scheduling: `ENG101 ≠ Mon-AM`

This simply removes one value from the domain of a single variable.

Binary Constraints

A binary constraint involves two variables.

Examples:

- In map coloring: neighboring regions must have different colors
- In exam scheduling: `CS101 ≠ CS102`

Higher-Order Constraints

A higher-order constraint involves three or more variables. For example, a scheduling rule like “at most two exams may occur in the same slot” involves multiple variables together.

Soft Constraints

Soft constraints express preferences rather than absolute rules. For example, the university may prefer `MATH201` in the morning, but this preference may be violated if needed.

When preferences are added, the task becomes closer to constrained optimization rather than pure satisfaction.

> **Things to Note:**
>
> - Unary constraints reduce a domain directly.
> - Binary constraints are especially important because many CSP algorithms are designed around them.
> - Soft constraints do not make an assignment invalid; they make some valid assignments better than others.

6. Why CSP Representation Is Powerful

The main strength of CSPs is not just that they define a problem clearly. The real strength is that this representation allows special solving techniques that exploit structure.

Once a problem is written as a CSP, we can use methods such as:

- backtracking search,
- smart variable and value ordering heuristics,
- forward checking,

- constraint propagation,
- arc consistency,
- problem decomposition based on graph structure.

These methods are far more effective than blindly exploring possibilities.

> **Key Insight:**
>
> CSP representation does not remove difficulty by magic. It removes unnecessary blindness. It exposes structure so that algorithms can reason more intelligently.

7. What Counts as a Solution?

Students often think that a solution is any partial progress toward the goal. In CSPs, that is not enough.

A valid solution must satisfy two conditions:

- every variable must be assigned a value,
- all hard constraints must be satisfied.

A partial assignment may be promising, but it is not a final solution.

Example

If in exam scheduling we assign:

- `CS101 = Mon-AM`
- `CS102 = Tue-AM`

that is only a partial assignment. It may be consistent so far, but it is not yet a solution until all remaining exams are assigned valid slots as well.

> **Common Confusion:**
>
> Do not confuse a *consistent partial assignment* with a *solution*. A consistent partial assignment violates no constraints so far, but it may still fail later. A solution is complete and fully valid.

8. Things to Think About

- If the path does not matter in a CSP, why is this useful when designing search algorithms?
- Why is exam scheduling a better example than map coloring for understanding soft constraints?
- If one variable has no legal values left, what does that tell you about the whole problem or current branch?
- Why do structured representations often lead to better algorithms?

9. Quick Recap

- A CSP consists of variables, domains, and constraints.
- A solution is a complete assignment that satisfies all hard constraints.
- CSPs differ from standard search because they expose internal structure.
- Map coloring and exam scheduling are both CSPs, even though they look very different.
- Constraints may be unary, binary, higher-order, or soft.
- The main power of CSPs comes from the fact that specialized algorithms can exploit problem structure.

10. Part 1 Revision Lines

> - Remember the CSP formula: **Variables + Domains + Constraints**.
> - Remember the solution condition: **complete and constraint-satisfying**.
> - Remember the big idea: **CSP is useful because structure becomes visible**.

End of Part 1